Introduction to Database Management System (Unit-1)

As the name suggests, the database management system consists of two parts. They are:

1. Database and
2. Management System

What is a Database?

A database is an organized collection of structured or unstructured data that can be accessed, managed, and updated by a computer system.

Data refers to a collection of facts, observations, measurements, or other information that can be processed or analyzed to gain insights, draw conclusions, or make decisions. Data can be in various forms, including numbers, text, images, audio, video, or other formats.

A database in a DBMS could be viewed by lots of different people with different responsibilities.

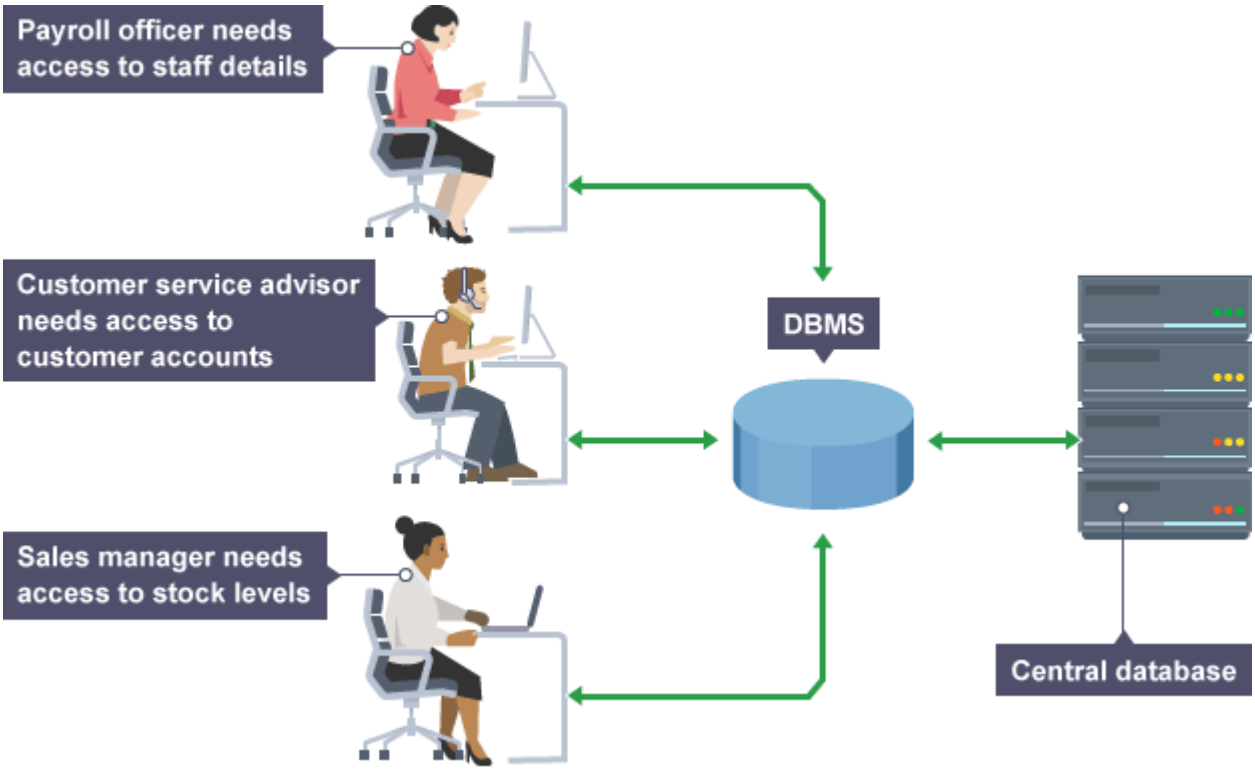


Figure 1.1: Employees are accessing Data through DBMS

For example, within a company there are different departments, as well as customers, who each need to see
different kinds of data. Each employee in the company will have different levels of access to the database with

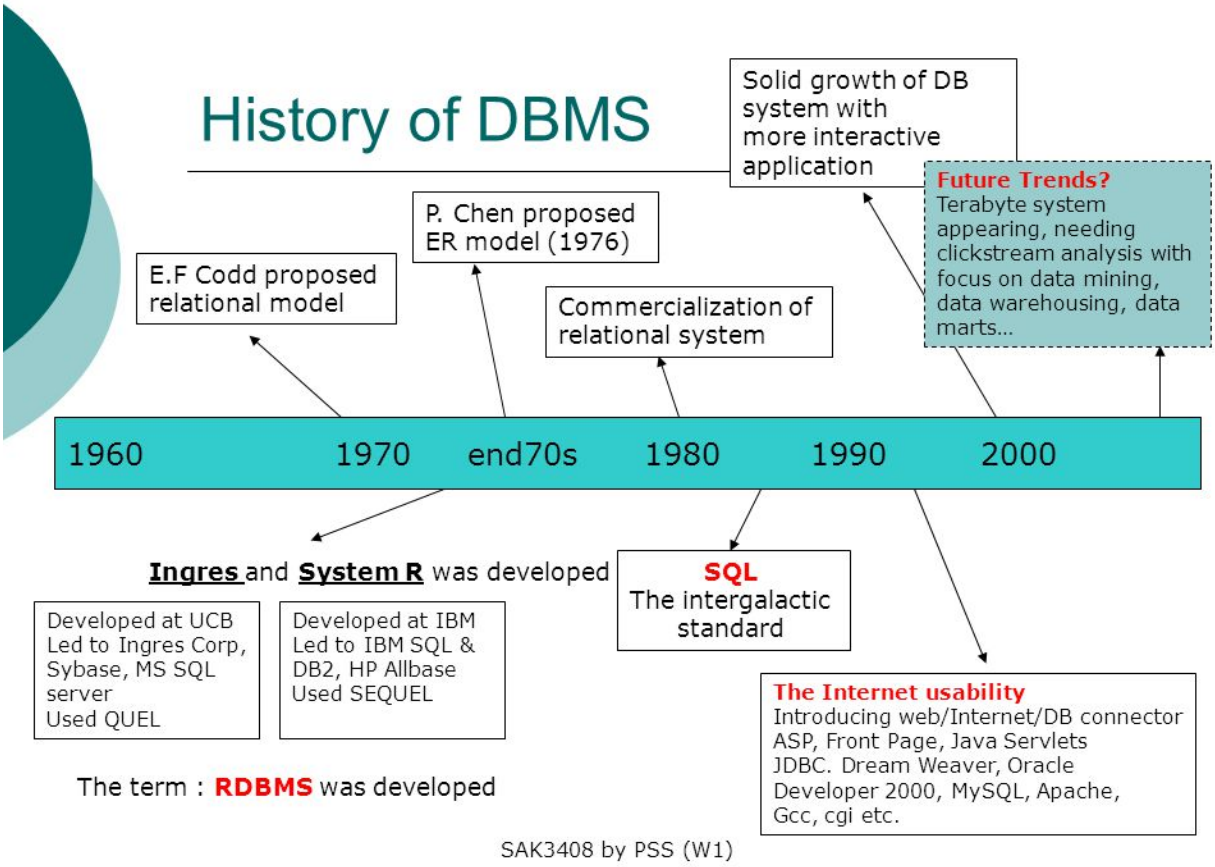
their own customized **front-end** application.
In a database, data is organized strictly in row and column format. The rows are called **Tuple** or **Record**. The
data items within one row may belong to different data types. On the other hand, the columns are often called
Domain or **Attribute**. All the data items within a single attribute are of the same data type.

What is Database Management System?

A database-management system (DBMS) is a collection of interrelated data and a set of programs to access those data. This is a collection of related data with an implicit meaning and hence is a database. The collection of data, usually referred to as the database, contains information relevant to an enterprise. The primary goal of a DBMS is to provide a way to store and retrieve database information that is both convenient and efficient. By data, we mean known facts that can be recorded and that have implicit meaning.

The management system is important because without the existence of some kind of rules and regulations it is not possible to maintain the database. We have to select the particular attributes which should be included in a particular table; the common attributes to create relationship between two tables; if a new record has to be inserted or deleted then which tables should have to be handled etc. These issues must be resolved by having some kind of rules to follow in order to maintain the integrity of the database.

Database systems are designed to manage large bodies of information. Management of data involves both defining structures for storage of information and providing mechanisms for the manipulation of information. In addition, the database system must ensure the safety of the information stored, despite system crashes or attempts at unauthorized access. If data are to be shared among several users, the system must avoid possible anomalous results.



Database Management System (DBMS) and Its Applications:

A Database management system is a computerized record-keeping system. It is a repository or a container for collection of computerized data files. The overall purpose of DBMS is to allow he users to define, store, retrieve and update the information contained in the database on demand. Information can be anything that is of significance to an individual or organization.

Databases touch all aspects of our lives. Some of the major areas of application are as follows:

1. Banking

2. Airlines

3. Universities

4. Manufacturing and selling

5. Human resources & so on

◦ Sales: For customer, product, and purchase information.

◦ Accounting: For payments, receipts, account balances, assets and other accounting information.

◦ Human resources: For information about employees, salaries, payroll taxes, and benefits, and for generation of paychecks.

◦ Manufacturing: For management of the supply chain and for tracking production of items in factories, inventories of items in warehouses and stores, and orders for items.

Online retailers: For sales data noted above plus online order tracking generation of recommendation lists, and maintenance of online product evaluations.

◦ Banking: For customer information, accounts, loans, and banking transactions.

◦ Credit card transactions: For purchases on credit cards and generation of monthly statements.

◦ Finance: For storing information about holdings, sales, and purchases of financial instruments such as stocks and bonds; also for storing real-time market data to enable online trading by customers and automated trading by the firm.

• Universities: For student information, course registrations, and grades (in addition to standard enterprise information such as human resources and accounting).

• Airlines: For reservations and schedule information. Airlines were among the first to use databases in a geographically distributed manner.

• Telecommunication: For keeping records of calls made, generating monthly bills, maintaining balances on prepaid calling cards, and storing information about the communication networks.

Database System vs File System

File System	DBMS
• **A file system is a software that manages and organizes the files in a storage medium. It controls how data is stored and retrieved.**	• DBMS or Database Management System is a software application. It is used for accessing, creating, and managing databases.
• **The file system provides the details of data representation and storage of data.**	• DBMS gives an abstract view of data that hides the details
• **Storing and retrieving of data can't be done efficiently in a file system.**	• DBMS is efficient to use as there are a wide variety of methods to store and retrieve data.
• **It does not offer data recovery processes.**	• There is a backup recovery for data in DBMS.
• **The file system doesn't have a crash recovery mechanism.**	• DBMS provides a crash recovery mechanism
• **Protecting a file system is very difficult.**	• DBMS offers good protection mechanism.
• **In a file management system, the redundancy of data is greater.**	• The redundancy of data is low in the DBMS system.

//Read More

Purpose of Database Systems

1. **Data redundancy and inconsistency**
   Since different programmers create the files and application programs over a long period, the various files are likely to have different structures and the programs may be written in several programming languages. Moreover, the same information may be duplicated in several places (files).

   For example, if a student has a double major (say, music and mathematics) the address and telephone number of that student may appear in a file that consists of student records of students in the Music department and in a file that consists of student records of students in the Mathematics department. This redundancy leads to higher storage and access cost. In addition, it may lead to data inconsistency; that is, the various copies of the same data may no longer agree. For example, a changed student address may be reflected in the Music department records but not elsewhere in the system.

2. **Difficulty in accessing data**.
   Difficulty in accessing data refers to situations where users or applications are unable to retrieve or use data stored in a database system.

   Suppose that one of the university clerks needs to find out the names of all students who live within a particular postal-code area. The clerk asks the data-processing department to generate such a list. Because the designers of the original system did not anticipate this request, there is no application program on hand to meet it. There is, however, an application program to generate the list of all students. The university clerk has now two choices: either obtain the list of all students and extract the needed information manually or ask a programmer to write the necessary application program. Both alternatives are obviously unsatisfactory. Suppose that such a program is written, and that, several days later, the same clerk needs to trim that list to include only those students who have taken at least 60 credit hours. As expected, a program to generate such a list does not exist. Again, the clerk has the preceding two options, neither of which is satisfactory. The point here is that conventional file-processing environments do not allow needed data to be retrieved in a convenient and efficient manner. More responsive data-retrieval systems are required for general use.
3. **Data isolation**
   Because data are scattered in various files, and files may be in different formats, writing new application programs to retrieve the appropriate data is difficult. Data isolation is a property of database systems that ensures that one transaction does not interfere with another transaction and also different data (formats) from different places (location) is causes hard to retrieve an appropriate data.
4. **Integrity problems**
   The data values stored in the database must satisfy certain types of consistency constraints. Integrity problems in a database system refer to situations where the data stored in the database is incorrect, inconsistent, or violates some predefined rules or constraints. (The integrity of data refers to the accuracy, completeness, and consistency of data in a database.)

Suppose the university maintains an account for each department, and records the balance amount in each account. Suppose also that the university requires that the account balance of a department may never fall below zero. Developers enforce these constraints in the system by adding appropriate code in the various application programs. However, when new constraints are added, it is difficult to change the programs to enforce them. The problem is compounded when constraints involve several data items from different files.

5. **Atomicity problems**
   A transaction either completes successfully and commits all its changes, or it fails and rolls back all its changes, leaving the database in its original state. A computer system, like any other device, is subject to failure. In many applications, it is crucial that, if a failure occurs, the data be restored to the consistent state that existed prior to the failure.

Consider a program to transfer $500 from the account balance of department A to the account balance of department B. If a system failure occurs during the execution of the program, it is possible that the $500 was removed from the balance of department A but was not credited to the balance of department B, resulting in an inconsistent database state. Clearly, it is essential to database consistency that either both the credit and debit occur, or that neither occur.

That is, the funds transfer must be atomic—it must happen in its entirety or not at all. It is difficult to ensure atomicity in a conventional file-processing system.

6. **Concurrent-access anomalies**
   Concurrent access refers to the ability of multiple users or applications to access a database system simultaneously. For the sake of overall performance of the system and faster response, many systems allow multiple users to update the data simultaneously.

   Indeed, today, the largest Internet retailers may have millions of accesses per day to their data by shoppers. In such an environment, interaction of concurrent updates is possible and may result in inconsistent data. Consider department A, with an account balance of $10,000. If two department clerks debit the account balance (by say $500 and $100, respectively) of department A at almost exactly the same time, the result of the concurrent executions may leave the budget in an incorrect (or inconsistent) state. Suppose that the programs executing on behalf of each withdrawal read the old balance, reduce that value by the amount being withdrawn, and write the result back. If the two programs run concurrently, they may both read the value $10,000, and write back $9500 and $9900, respectively. Depending on which one writes the value last, the account balance of department A may contain either $9500 or $9900, rather than the correct value of $9400. To guard against this possibility, the system must maintain some form of supervision. But supervision is difficult to provide because data may be accessed by many different application programs that have not been coordinated previously.
7. **Security problems**
   Not every user of the database system should be able to access all the data. Database systems are vulnerable to a variety of security problems that can compromise the confidentiality, integrity, and availability of the data they manage.

   For example, in a university, payroll personnel need to see only that part of the database that has financial information. They do not need access to information about academic

records. But, since application programs are added to the file-processing system in an ad hoc manner, enforcing such security constraints is difficult. These difficulties, among others, prompted the development of database systems. In what follows, we shall see the concepts and algorithms that enable database systems to solve the problems with file-processing systems

8. **Supports multiple views of data**

   Supporting multiple views of data is an important feature of a database system that allows different users or applications to access the same data in different ways, depending on their specific needs.

Advantages of DBMS:

Controlling of Redundancy: Data redundancy refers to the duplication of data (i.e storing same data multiple times). In a database system, by having a centralized database and centralized control of data by the DBA the unnecessary duplication of data is avoided. It also eliminates the extra time for processing the large volume of data. It results in saving the storage space.

Improved Data Sharing : DBMS allows a user to share the data in any number of application programs.

Data Integrity : Integrity means that the data in the database is accurate. Centralized control of the data helps in permitting the administrator to define integrity constraints to the data in the database.
For example: in customer database we can can enforce an integrity that it must accept the customer only from Noida and Meerut city.

Security : Having complete authority over the operational data, enables the DBA in ensuring that the only mean of access to the database is through proper channels. The DBA can define authorization checks to be carried out whenever access to sensitive data is attempted.

Data Consistency : By eliminating data redundancy, we greatly reduce the opportunities for inconsistency.

For example: is a customer address is stored only once, we cannot have disagreement on the stored values. Also updating data values is greatly simplified when each value is stored in one place only. Finally, we avoid the wasted storage that results from redundant data storage.

Efficient Data Access: In a database system, the data is managed by the DBMS and all access to the data is through the DBMS providing a key to effective data processing

Enforcements of Standards: With the centralized of data, DBA can establish and enforce the data standards which may include the naming conventions, data quality standards etc.

Data Independence : Ina database system, the database management system provides the interface between the application programs and the data. When changes are made to the data representation, the meta data obtained by the DBMS is changed but the DBMS is continues to

provide the data to application program in the previously used way. The DBMs handles the task of transformation of data wherever necessary.
IT can be defined as the capacity to change the schema at one level of a database system without having
to change schema at the next higher level. There are mainly two types of Data Independence
1. Logical data independence: The ability to change the logical (conceptual) schema without changing the External schema (User View) is called logical data independence. For example, the addition or removal of new entities, attributes, or relationships to the conceptual schema should be possible without having to change existing external schemas or having to rewrite existing application programs.
2. Physical data independence: The ability to change the physical schema without changing the logical schema is called physical data independence. For example, a change to the internal schema, such as using different file organization or storage structures, storage devices, or indexing strategy, should be possible without having to change the conceptual or external schemas.

Reduced Application Development and Maintenance Time : DBMS supports many important functions that are common to many applications, accessing data stored in the DBMS, which facilitates the quick development of application.

Disadvantages of DBMS
1) It is bit complex. Since it supports multiple functionality to give the user the best, the underlying software
has become complex. The designers and developers should have thorough knowledge about the software
to get the most out of it.
2) Because of its complexity and functionality, it uses large amount of memory. It also needs large memory to run efficiently.
3) DBMS system works on the centralized system, i.e.; all the users from all over the world access this
database. Hence any failure of the DBMS, will impact all the users.
4) DBMS is generalized software, i.e.; it is written work on the entire systems rather specific one. Hence some of the application will run slow.

View of Data

A database system is a collection of interrelated data and a set of programs that allow users to access and modify these data. A major purpose of a database system is to provide users with an abstract view of the data. That is, the system hides certain details of how the data are stored and maintained.

Data Abstraction

For the system to be usable, it must retrieve data efficiently. The need for efficiency has led designers to use complex data structures to represent data in the database. Since many database-

system users are not computer trained, developers hide the complexity from users through several levels of abstraction, to simplify users' interactions with the system.

So, Data abstraction is the process of hiding complex details and presenting only essential information to users.

Physical level: This is the lowest level of abstraction and describes how the data is stored on the storage media such as hard disks, solid-state drives, or magnetic tapes. It deals with details such as data organization, access methods, and storage structures.

Logical level: This level of abstraction describes how the data is organized and structured in a database. It provides a conceptual view of the data and is concerned with the relationships between entities, attributes, and constraints.

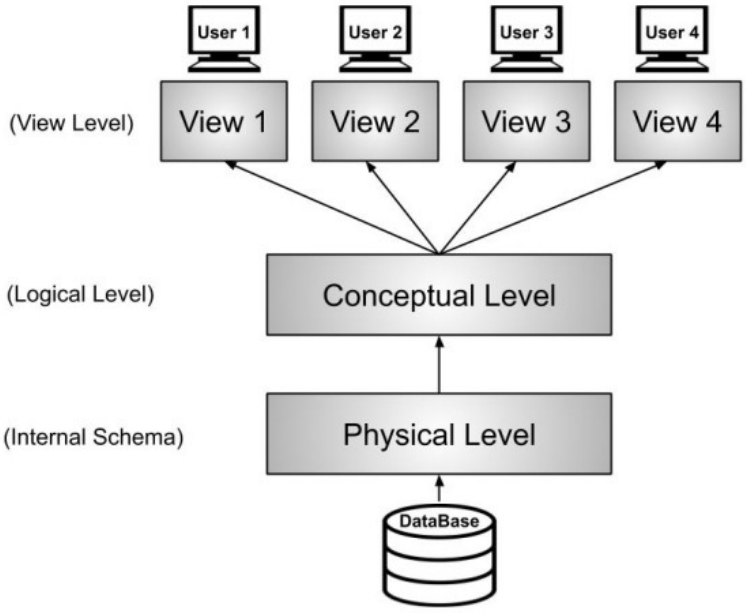
View level: This is the highest level of abstraction and describes how the data is presented to users. It provides a user-specific view of the data and is concerned with the selection, projection, and restriction of data.

Instances and Schemas

Databases change over time as information is inserted and deleted. The collection of information stored in the database at a particular moment is called an instance of the database. The overall design of the database is called the database schema. Schemas are changed infrequently, if at all. The concept of database schemas and instances can be understood by analogy to a program written in a programming language. A database schema corresponds to the variable declarations (along with associated type definitions) in a program.

Each variable has a particular value at a given instant. The values of the variables in a program at a point in time correspond to an instance of a database schema. Database systems have several schemas, partitioned according to the levels of abstraction. The physical schema describes the database design at the physical
level, while the logical schema describes the database design at the logical level. A database may also have several schemas at the view level, sometimes called subschemas, which describe different views of the database. Of these, the logical schema is by far the most important, in terms of its effect on application programs, since programmers construct applications by using the logical schema. The physical schema is hidden beneath the logical schema, and can usually be changed easily without affecting application programs. Application programs are said to exhibit physical data independence if they do not depend on the physical schema, and thus need

not be rewritten if the physical schema changes.



Levels of Data Abstraction

Physical or Internal Level

The levels of abstraction within a Database Management System (DBMS) offer various advantages to users and applications. The physical level, the lowest tier of data abstraction, delineates the storage mechanisms employed within the database. It encompasses data structures utilised for storage and the methods employed to access stored data.

Due to its complexity, this level is typically obscured from users. Database administrators hold the responsibility of determining the specifics of data storage within the database, encompassing aspects such as data organisation, allocation of disk space, and methods for data access.

Logical or Conceptual Level

The logical level, situated between the physical and conceptual tiers, delineates the data housed within the database and the relationships existing among them. This level is relatively less intricate when contrasted with the physical tier. Programmers typically operate within this realm, where decisions regarding data structures, table organisation, relationships, and constraints are made based on the nature of the data.

View or External Level

At the pinnacle of abstraction lies the conceptual level. Within this tier, various views are established, with each offering a partial representation of the complete dataset tailored to the user's needs. This level of data abstraction in DBM defines multiple views of the database to simplify user interaction, providing a comprehensible overview of the data. As the highest tier of abstraction, the conceptual level is designed to be the most user-friendly and easily understandable.

Data Models

The data model is a collection of conceptual tools for describing data, data relationships, data semantics, and consistency constraints. A data model provides a way to describe the design of a database at the physical, logical, and view levels. Data relationships define the connections and associations between different data entities in a database. Data semantics refers to the meaning and interpretation of the data in a database. Consistency constraints are rules and restrictions that govern the data in a database and ensure its accuracy and integrity. A data model provides a way to describe the design of a database at the physical, logical, and view levels.

The data models can be classified into four different categories:

Relational Model

The relational model uses a collection of tables to represent both data and the relationships among those data. Each table has multiple columns, and each column has a unique name. Tables are also known as relations. The relational model is an example of a record-based model. Record-based models are so named because the database is structured in fixed-format records of several types. Each table contains records of a particular type. Each record type defines a fixed number of fields, or attributes. The columns of the table correspond to the attributes of the record type. The relational data model is the most widely used data model, and a vast majority of current database systems are based on the relational model. Entity-Relationship Model. The entity-relationship (E-R) data model uses a collection of basic objects, called entities, and relationships among these objects. An entity is a “thing” or “object” in the real world that is distinguishable from other objects. The entity relationship model is widely used in database design.

Object-Based Data Model

Object-oriented programming (especially in Java, C++, or C#) has become the dominant software-development methodology. This led to the development of an object-oriented data model that can be seen as extending the E-R model with notions of encapsulation, methods (functions), and object identity. The object-relational data model combines features of the object-oriented data model and relational data model.

Semi-structured Data Model

The semi-structured data model permits the specification of data where individual data items of the same type may have different sets of attributes. This is in contrast to the data models mentioned earlier, where every data item of a particular type must have the same set of attributes. The Extensible Markup Language (XML) is widely used to represent semi-structured data.

Database Languages

A database system provides a data-definition language to specify the database schema and a data-manipulation language to express database queries and updates. In practice, the data definition and data-manipulation languages are not two separate languages; instead they simply form parts of a single database language, such as the widely used SQL language.

Data-Manipulation Language

A data-manipulation language (DML) is a language that enables users to access or manipulate data as organized by the appropriate data model.
The types of access are:
• Retrieval of information stored in the database
• Insertion of new information into the database
• Deletion of information from the database
• Modification of information stored in the database
There are basically two types:
• Procedural DMLs require a user to specify what data are needed and how to get those data.
• Declarative DMLs (also referred to as nonprocedural DMLs) require a user to specify what data are needed without specifying how to get those data.

The DML component of the SQL language is nonprocedural.
A **query is a statement requesting** the retrieval of information. The portion of a DML that involves
information retrieval is called a **query language**. Although technically incorrect, it is common practice to
use the terms query language and data-manipulation language synonymously.
This query in the SQL language finds the name of the customer whose customer-id is 192-83-7465:
select customer.customer-name from customer where customer.customer-id = 192-83-7465.

Data-Definition Language (DDL)
We specify a database schema by a set of definitions expressed by a special language called a data-definition language (DDL). The DDL is also used to specify additional properties of the data.

We specify the storage structure and access methods used by the database system by a set of statements in a special type of DDL called a data storage and definition language. These statements define the implementation details of the database schemas, which are usually hidden from the users. The data values stored in the database must satisfy certain consistency constraints.

DDL	DML
DDL stands for Data Definition Language.	DML stands for Data Manipulation Language.
It helps us to define the structure of the database.	It helps us to manage the data.
It affects the whole table.	It only affects the specified row(s) of the table.
It helps in defining the fields or columns of the tables.	It helps us in defining rows or records of the tables.
They are auto-committed (changes made are permanent).	They are not auto-committed (changes are not permanent).
Changes are permanent.	We can roll back our changes in the DML language
They don't have further classification.	They are classified into procedural and non-procedural.
Commands used: CREATE, ALTER, TRUNCATE, RENAME, DROP.	Commands used: SELECT, UPDATE, DELETE, INSERT.

Data control language (DCL) is used to access the stored data. It is mainly used for revoke and to grant the user the required access to a database. In the database, this language does not have the feature of rollback.
Some Examples:
Grant- gives user's access privileges to database.
Revoke- Withdraw access privileges given with GRANT command.

Database Administrators and Database Users

A primary goal of a database system is to retrieve information from and store new information in the database. People who work with a database can be categorized as database users or database administrators.

Database User

There are four different types of database-system users, differentiated by the way they expect to interact with the system.
Naive users are unsophisticated users who interact with the system by invoking one of the application programs that have been written previously.

Application programmers are computer professionals who write application programs. Application programmers can choose from many tools to develop user interfaces. Rapid application development (RAD)tools are tools that enable an application programmer to construct forms and reports without writing a program.

Sophisticated users interact with the system without writing programs. Instead, they form their requests in a database query language.

Database Administrator

One of the main reasons for using DBMSs is to have central control of both the data and the programs that access those data. A person who has such central control over the system is called a database administrator (DBA).

The functions of a DBA include:

1. Schema definition. The DBA creates the original database schema by executing a set of data definition statements in the DDL.

2. Storage structure and access-method definition. The DBA may specify some parameters pertaining to the physical organization of the data and the indices to be created.

3. Schema and physical-organization modification. The DBA carries out changes to the schema and physical organization to reflect the changing needs of the organization, or to alter the physical organization to improve performance.

4. Granting of authorization for data access. By granting different types of authorization, the database administrator can regulate which parts of the database various users can access. The authorization information is kept in a special system structure that the database system consults whenever a user tries to access the data in the system.
5. Routine maintenance. Examples of the database administrator's routine maintenance activities are:
    - Periodically backing up the database onto remote servers, to prevent loss of data in case of disasters such as flooding.
    - Ensuring that enough free disk space is available for normal operations, and upgrading disk space as required.
    - Monitoring jobs running on the database and ensuring that performance is not degraded by very expensive tasks submitted by some users.

Database and Application Architecture

1. System Architecture

We are now in a position to provide a single below picture of the various components of a database system and the connections among them.

The architecture of a database system is greatly influenced by the underlying computer system on which the database system runs. Database systems can be centralized, or client-server, where one server machine executes work on behalf of multiple client machines. Database systems can also be designed to exploit parallel computer architectures. Distributed databases span multiple geographically separated machines.

The database system is divided into three components: Query Processor, Storage Manager, and Disk Storage.

1. **Query Processor**: It interprets the requests (queries) received from end user via an application program into instructions. It also executes the user request which is received from the DML compiler.

Query Processor contains the following components

- DML Compiler: It processes the DML statements into low level instruction (machine language), so that they can be executed.
- DDL Interpreter: It processes the DDL statements into a set of table containing meta data (data about data).
- Embedded DML Pre-compiler: It processes DML statements embedded in an application program into procedural calls.
- Query Optimizer: It executes the instruction generated by DML Compiler.

2. Storage Manager: Storage Manager is a program that provides an interface between the data stored in the database and the queries received. It is also known as Database Control System. It maintains the consistency and integrity of the database by applying the constraints and executing the DCL statements. It is responsible for updating, storing, deleting, and retrieving data in the database.

It contains the following components –

- **Authorization Manager**: It ensures role-based access control, i.e,. checks whether the particular person is privileged to perform the requested operation or not.
- **Integrity Manager:** It checks the integrity constraints when the database is modified.
- **Transaction Manager:** It controls concurrent access by performing the operations in a scheduled way that it receives the transaction. Thus, it ensures that the database remains in the consistent state before and after the execution of a transaction.
- **File Manager:** It manages the file space and the data structure used to represent information in the database.
- **Buffer Manager**: It is responsible for cache memory and the transfer of data between the secondary storage and main memory.

3. Disk Storage:

It contains the following components –

- Data Files: It stores the data.
- Data Dictionary: It contains the information about the structure of any database object. It is the repository of information that governs the metadata.
- Indices: It provides faster retrieval of data item.
- The structure of a Database Management System (DBMS) can be divided into three main components: the Internal Level, the Conceptual Level, and the External Level.

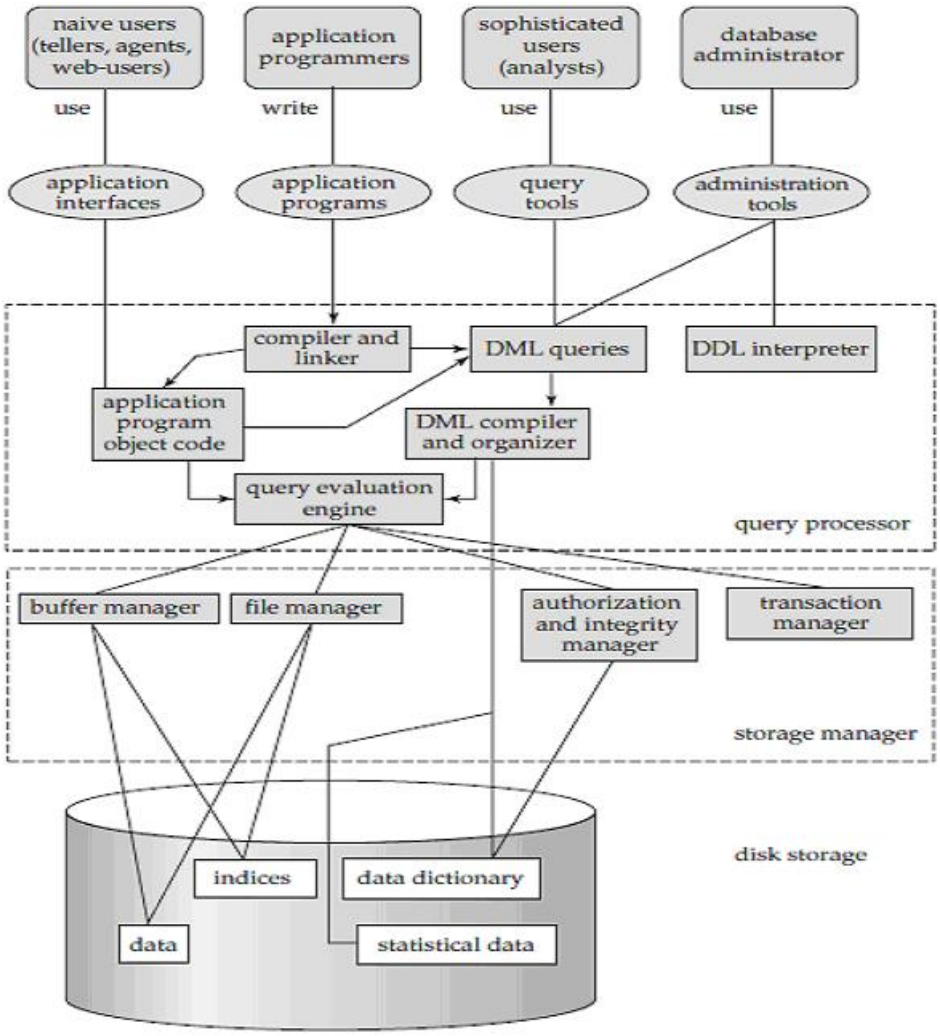


Fig:System Architecture

2.Database Application Architecture

- The DBMS design depends upon its architecture. The basic client/server architecture is used to deal with a large number of PCs, web servers, database servers and other components that are connected with networks.
- The client/server architecture consists of many PCs and a workstation which are connected via the network.
- DBMS architecture depends upon how users are connected to the database to get their request done.

1 Tier Architecture in DBMS is the simplest architecture of Database in which the client, server, and Database all reside on the same machine. A simple one tier architecture example would be anytime you install a Database in your system and access it to practice SQL queries. But such architecture is rarely used in production.

A **2 Tier Architecture** in DBMS is a Database architecture where the presentation layer (Interface) runs on a client (PC, Mobile, Tablet, etc.), and data is stored on a server called the second tier. Two tier architecture provides added security to the DBMS as it is not exposed to the end-user directly. It also provides direct and faster communication. It invokes database system functionality at the server machine through query language statements. The client application is responsible for user interface and business logic, while the database server handles data storage and retrieval. This architecture is also known as client-server architecture.

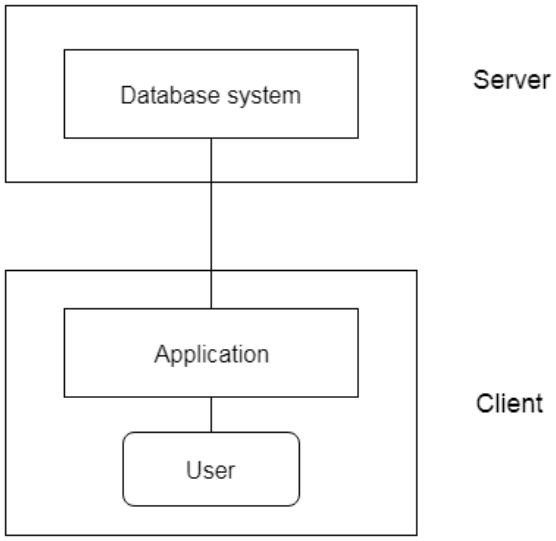


Fig:2 tier Architecture

A **3 Tier Architecture** in DBMS is the most popular client server architecture in DBMS in which the development and maintenance of functional processes, logic, data access, data storage,

and user interface is done independently as separate modules. Three Tier architecture contains a presentation layer, an application layer, and a database server.

3-Tier database Architecture design is an extension of the 2-tier client-server architecture. A 3-tier architecture has the following layers:

- Presentation layer (your PC, Tablet, Mobile, etc.)
  - User Client interface from where user interact or simply application
- Application layer (server)
  - It resides between the user and the DBMS, which is responsible for communicating the user's request to the DBMS system and send the response from the DBMS to the user.
- Database Server
  - It processes functional logic, constraint, and rules before passing data to the user or down to the DBMS.
  -

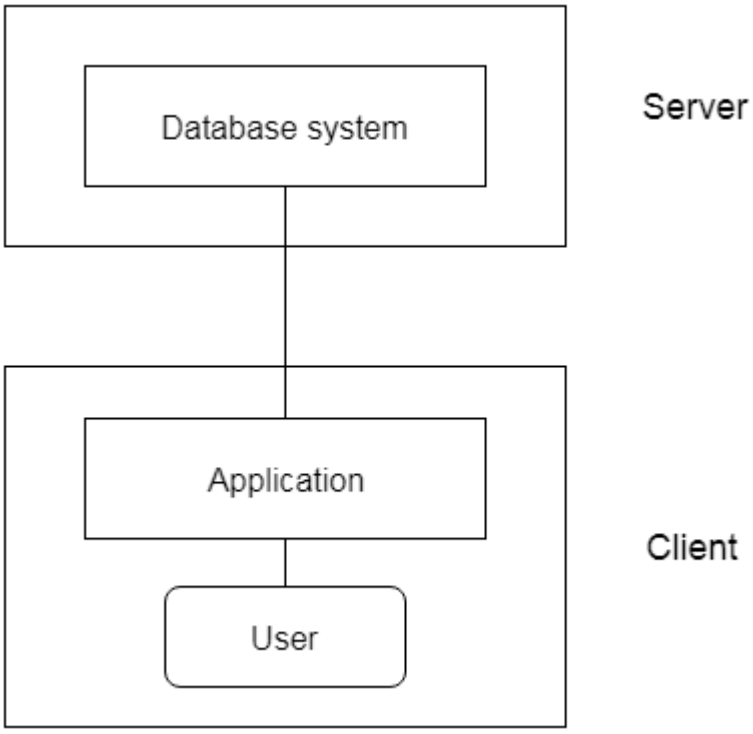


Fig:-3 Tier Architecture

//Read Advantages and Disadvantages of each tier